

Sistema de gestión de tareas de un servidor web mediante la simulación de un sistema Productor-Consumidor

Sistemas Operativos

07 de Junio de 2025

1. Descripción del Problema

Se te ha asignado la creación de un sistema de gestión de tareas para un servidor web de alto rendimiento llamado "Sistope-Server". Este sistema debe ser capaz de gestionar una alta demanda de peticiones de usuarios de forma concurrente.

En este sistema, múltiples Hilos Receptores (Productores) se encargarán de aceptar conexiones entrantes y generar "tareas" (como procesar una petición HTTP, consultar una base de datos, etc.). Estas tareas serán colocadas en una cola de tareas. Al mismo tiempo, un "pool" de Hilos Trabajadores (Consumidores) estará encargado de recoger las tareas de la cola y procesarlas.

El objetivo será desarrollar un programa en C que simule este flujo de trabajo a través del uso de hebras y semáforos. Para ello se debe garantizar que este sistema funcione de manera eficiente, sin perder tareas ni generar condiciones de carrera.

2. Requisitos Funcionales

1. **Configuración:** El programa deberá utilizar los siguientes (`#define`) para su elaboración:

- `TAMANO_BUFFER`: El número máximo de tareas que pueden estar en la cola de espera.
- `NUM_RECEPTORES`: El número de hilos receptores que van a producir tareas.
- `NUM TRABAJADORES`: El número de hilos trabajadores que van a consumir estas tareas.
- `TOTAL_TAREAS`: La cantidad total de tareas a generar.

2. **Estructura de Datos Compartida:**

- Se debe implementar un **arreglo circular** que actuará como buffer compartido (cola de tareas) para almacenar las **tareas**. Cada tarea será representada por un número entero que simulará su ID de petición.

3. **Procesamiento Concurrente:**

- El programa principal creará `NUM_RECEPTORES` hebras productoras y `NUM TRABAJADORES` hebras consumidoras utilizando `pthread_create`.

4. **Lógica de Receptores y Trabajadores:**

- **Receptores (Productores):** Cada hebra receptora generará IDs de tareas a través de un número entero y los depositará en la cola de tareas.
- **Trabajadores (Consumidores):** Cada hebra trabajadora extraerá una tarea de la cola de tareas y simulará su procesamiento imprimiendo un mensaje en la consola (ej: `Trabajador 2 procesó la tarea 152`).

5. **Sincronización con Semáforos:**

- Se deberá implementar y utilizar **semáforos** para proteger el acceso a la cola de tareas (cola compartida) y coordinar a los **receptores** y **trabajadores**.

- Se requiere un semáforo para contar los **espacios vacíos** en la cola (**empty**).
 - Se requiere un semáforo para contar las **tareas pendientes** en la cola (**full**).
 - Se requiere un semáforo binario (**mutex**) para garantizar el **acceso mutuamente excluyente** a la estructura de la cola de tareas al momento de insertar o extraer una **tarea**.
6. **Finalización de Hebras:** La hebra principal debe esperar a que todas las hebras receptoras y trabajadoras terminen su ejecución utilizando `pthread_join`. Una vez finalizadas, el programa debe mostrar un mensaje de resumen indicando que todas las **tareas** fueron procesadas.

3. Evaluación

La tarea será evaluada en base a los siguientes criterios:

- **Documentación y Claridad del Código:** Comentarios, nombres de variables y funciones descriptivos, y una estructura general clara.
- **Generación de hebras:** Las hebras se generan de manera correcta, las definiciones de funciones y parámetros de entrada son correctos.
- **Comunicación de hebras:** Las hebras se comunican de forma correcta usando variables en memoria global (buffer) y se protegen adecuadamente.
- **Concurrencia:** Las hebras creadas trabajan concurrentemente entre ellas.
- **Simulación:** La simulación y salida del sistema de productores-consumidores es correcta.

4. Resultados Esperados (Salida)

Dado que el sistema es concurrente, el orden exacto de la salida en la consola variará con cada ejecución. Los mensajes de los receptores y trabajadores aparecerán intercalados, demostrando que están operando en paralelo.

Un extracto de la salida (suponiendo `TOTAL.TAREAS = 1000`) podría verse de la siguiente manera:

```
Receptor 0 produjo la tarea 1
Receptor 1 produjo la tarea 2
Receptor 0 produjo la tarea 3
Trabajador 0 proceso la tarea 1
Receptor 2 produjo la tarea 4
Trabajador 1 proceso la tarea 2
Receptor 1 produjo la tarea 5
Trabajador 0 proceso la tarea 3
...
... (producción y procesamiento de otras tareas) ...
...
Trabajador 3 proceso la tarea 998
Receptor 0 produjo la tarea 999
Receptor 2 produjo la tarea 1000
Trabajador 1 proceso la tarea 999
Trabajador 2 proceso la tarea 1000
```

Una vez que todas las `TOTAL.TAREAS` hayan sido generadas y procesadas por completo, el programa principal debe imprimir el mensaje final de resumen, demostrando que esperó a todas las hebras.

```
Todas las 1000 tareas han sido procesadas.
Finalizando el programa.
```

Plazo de Entrega

Fecha Límite de Entrega: 06 de Diciembre de 2025 hasta las 23:59 hrs.